



Implementing DDD Workshop

Student Handout

Thank you for joining us for the Implementing Domain-Driven Design workshop! We look forward to getting to know you over the course of the workshop.

Quick Details

- This is a hands-on workshop.
- Students are strongly encouraged to engage in workshop exercises and write source code that includes implementing multiple interacting Microservices, as DDD Bounded Contexts, that are event-driven.
- You will use DDD strategic and tactical design in your implementations.
- Exercises are worked within a Peer-to-Peer Economy business, involving e-commerce in a whole new light.

Online, live teaching, 4 half-days

This live online workshop is fully redesigned for the best online experience and includes all the great content of the best-selling on-site training. Break up your day with a morning or afternoon of DDD training and then get back to your regular development work.

- Set aside the time that is needed for the workshop without interruptions from your work. The focus you give to this time of learning will pay off.

Preparation Reminders

- Get your rest before the workshop.
- Bring your laptop with your development setup.
- Read [Domain-Driven Design Distilled](#). It should not take too long and will give you a good start to DDD concepts. Code: AWSS-VERNON saves 35% off the list price.

What you will learn

Day-1 Strategic DDD

The goal of Day 1 is for you to get grounded with a DDD mentality that will free your mind from data-driven approaches. This is necessary to grasp the thrust of developing a Ubiquitous Language

within a Bounded Context, the hallmark of DDD. From there you will be ready to embrace strategic design using Sub-domains, Bounded Contexts, and Context Maps.

Getting Started With DDD

Understand the essence of DDD. Form the right mentality for powerful software modeling. Develop a Ubiquitous Language for your project using the Specification by Example approach and Acceptance Tests. This lays a foundation for moving forward with DDD.

Domains, Subdomains, and Bounded Contexts

Grasp the core concepts of DDD strategic modeling, and put them to use. Learn to vigorously gather core concepts and separate irrelevant ones. Learn why strategic design is so essential. Develop the ability to determine when your domain models are composed correctly and when they require more rigorous design. You will gain a new tool from this experience, known as Context Topo Maps.

Context Mapping

Learn how to integrate with other subsystems using DDD and straightforward design techniques. Context Maps are not just drawings. Use them for cross-team communications, project thinking, learning, planning, and as an integration tool. You will use Context Topo Maps to make it all make sense.

Day-2 Architecture and Event-Driven Modeling

Day 2 crosses over to architecture, preparing you for modularity, to employ Ports and Adapters, Event-Driven, and other powerful architectural tools as you develop new Bounded Contexts in hands-on exercises and homework. Students learn ways to develop a Ubiquitous Language within a Bounded Context, using Event Storming, Extreme Modeling, and Context Topo Maps. Students use these tools to reach a shared understanding of a Core Domain and Subdomains.

Modular Architecture

If you don't understand modularity and apply it to the full, you will have trouble using DDD. Don't go to all the trouble to carefully craft a Ubiquitous Language with domain objects but overlook proper modularization. Learn why modularity at every level is so vital to good architecture and design. Capture the techniques for organizing concepts within a Bounded Context and inside its domain model.

Architectures That Support DDD

See how various architecture styles and patterns can be used to support your DDD projects. We will survey use of the following: Layers; Dependency Inversion Principle; Hexagonal; Microservices; REST; Event-Driven; Process Management; Event Sourcing and CQRS; Actor Model. Coupled with DDD, these architecture styles and patterns will support your microservices efforts.

Domain Events, Event Storming, Extreme Modeling

Learn to use Domain Events and Event Storming for both “big picture” systems designs and “detail level” tactical designs. Yet, with remote team interactions, you'll need a technique beyond Event Storming. That's why you'll learn to leverage Extreme Modeling. Includes online exercises.

Day-3 Tactical DDD

On Day 3 we switch our focus to tactical design, giving you the aptitude to make proper finer-grained modeling decisions within a Core Domain. Whether it's the use of Entities, Value Objects, Aggregates, Domain Events, or Domain Services that best fit your specific modeling situation, you can put each of the tactical tools to use in your own Core Domain project. Learn the advantages and tradeoffs of using each of the tools, and when another tool could provide a better set of trade offs.

Entities

Understand how to properly employ this necessary modeling tool. There will likely be at least some Entities in object-oriented domain models because of the need to manage change over the life cycle of mutable objects. Rather than overusing Entities, you will learn how to best use them only when a domain concept must change over time.

Value Objects

Embrace this invaluable, functional modeling tool, which you should use as often as possible. You will learn the Value Object characteristics and why using Values where possible is a safer way to design. You will learn by modeling concepts as Values, and you may even decide to use Value-prolific designs rather than Entities.

Aggregates

Use this powerful modeling tool to shape, structure, and compose transactional boundaries around a meaningful whole concept. Transform your understanding of Entity thinking into one that scales broadly and performs under high load, all while protecting valuable and time-sensitive business state.

Domain Services

Add powerful stateless operations to your model when its use is appropriate. The modeling techniques cover both domain-specific services and those of a more technical nature, often used for integration.

Factories

Learn when and how to use domain-specific factories to support the creation of domain objects.

Day-4 Strategic Design Meets Implementation

During Day 4 we bring all the parts together to form a microservice (or application). We look at different ways to persist Aggregates using object-relational mapping, document databases, key-value stores, as well as by employing Event Sourcing. We then take a deep dive into applying Context Mapping techniques as we return to strategic design, but this time by developing integration solutions between Core Domains and other Bounded Contexts in your whole-system solution. We give attention to other parts of the application, such as the user interface and querying with CQRS. We also take a look at how to support REST frontends and Applications Services when necessary. With all this background you will be prepared to move forward with a DDD project of your own, or to make greater strides on your current DDD strategic initiative.

Persistence and Event Sourcing

You will learn how to persist and retrieve your Aggregates using object-based persistence and with Event Sourcing.

CQRS

Learn what CQRS is and how to apply CQRS using full Entity state, Domain Events, and Event Sourced models. Discover how to partition your Bounded Context for two-sided operation for the command model and the query model. We consider the basics of query/read-side projections, how to deal with complex projections, and how to implement queries.

Integrating Bounded Contexts

Employ both REST and messaging to realize your Context Maps in code as you integrate multiple Bounded Contexts of enterprise applications. Understand the complexities and challenges of distributed computing environments, and how to deal with them. An important lesson that is reemphasized is to model each Bounded Context cleanly and separately from those with which you integrate. Integration techniques include long-running transactions by means of Process Managers, aka Sagas.

Application Patterns

Learn how the components that surround your domain model comprise a whole subsystem application. Consideration is given to User Interface strategies and the Application Programming Interface (API) using Application Services. Guidance includes how the CQRS query model factors in as well as using API composition.